

Universidade Federal do Rio Grande do Norte
Instituto Metr pole Digital
Programa de P s-Gradua  o em Tecnologia da Informa  o

Prof. Dr. Jean M rio Moreira de Lima
jean.lima@imd.ufrn.br

DESENVOLVIMENTO DE SOFTWARE COM IA

Atividade Pr tica Ass ncrona

Harness e Arquitetura na Pr tica

Informa  es gerais

Entrega: Link do reposit rio (GitHub) + relat rio em PDF/Markdown, conforme detalhado na Etapa 8

Prazo: 11/09/2026

Objetivo Geral

Voc  vai configurar controles reais sobre um agente de IA e, no mesmo projeto, revisar e documentar decis es de arquitetura como as que discutimos em aula.

O objetivo   testar, no seu pr prio projeto, mais de uma forma de resolver cada problema e registrar o que funcionou, o que n o funcionou, e por qu . Sempre que a atividade oferecer uma escolha ou pedir uma compara  o, invista tempo real nela:   nessa investiga  o que est  o principal aprendizado.

Etapa 1: Harness M nimo - Autonomia e Guardrail na Pr tica

Objetivo: configurar e comparar, na pr tica, controles reais de harness sobre um agente de IA.

Tarefas

1. Escolha um projeto: pode ser o mesmo iniciado nas pr ticas ass ncronas anteriores ou um novo reposit rio.
2. Escolha uma tarefa pequena e execute-a **duas vezes**, usando dois modos de permiss o/autonomia diferentes da sua ferramenta (ex.: plan mode vs. auto-accept edits). Registre, para cada execu  o: tempo gasto, sensa  o de controle e risco percebido.
3. Crie **1 hook (ou controle equivalente)** que bloqueie de fato uma a  o de risco espec fica do SEU projeto. Escolha um risco diferente do exemplo “bloquear merge na main” visto em aula.
4. Teste o hook: dispare a a  o bloqueada de prop sito e confirme que ele realmente impede a a  o.

Entreg vel desta etapa

- ☐ Compara  o escrita entre os dois modos de autonomia testados (tempo, controle, risco).
- ☐ Hook (ou controle equivalente) commitado no reposit rio, com print/log demonstrando o bloqueio funcionando.

Etapa 2: TDD como Guard-Rail

Objetivo: aplicar o ciclo Red-Green-Refactor e investigar uma ferramenta real de enforcement de TDD.

Tarefas

- Escolha 1 tarefa pequena e aplique o ciclo Red-Green-Refactor com o agente: escreva o teste antes de pedir a implementação.
- Investigue **UMA** ferramenta de enforcement automatizado como **tdd-guard** ou **Superpowers**. Instale-a se possível; se não for possível instalar (limitações de ambiente/acesso), leia a documentação oficial no repositório do projeto e documente, em detalhe, como ela se comportaria no seu cenário.
- Em uma tarefa separada, peça deliberadamente ao agente para implementar **SEM** escrever teste antes. Compare o resultado (qualidade, cobertura de casos de borda).

Entregável desta etapa

- ☐ Evidência do ciclo Red-Green-Refactor (histórico de commits ou log mostrando teste antes da implementação).
- ☐ Relato da investigação da ferramenta de enforcement escolhida (instalada ou documentada).
- ☐ Comparação escrita entre a tarefa feita com TDD e a tarefa feita sem TDD.

Etapa 3: Observabilidade e Checkpoint Humano

Objetivo: manter rastreabilidade do processo do agente e simular um ponto de parada obrigatório.

Tarefas

- Mantenha (ou exporte) o log/transcript da sessão do agente ao longo de toda a atividade. Não é necessário limpar ou editar.
- Defina **1 checkpoint humano obrigatório** para o seu projeto (ex.: antes de aplicar uma migração, antes de expor um novo endpoint, antes do merge) e simule-o: pare a execução no ponto definido, registre a decisão tomada (aprovar / editar / rejeitar).

Entregável desta etapa

- ☐ Log/transcript da sessão salvo em docs/sessao-log.md (ou anexo).
- ☐ Registro do checkpoint definido, da decisão tomada e do papel humano assumido (com justificativa).

Etapa 4: Revisão Arquitetural com Apoio de IA

Objetivo: aplicar, no seu próprio projeto, os princípios de modularidade, baixo acoplamento e contratos claros.

Tarefas

- Peça a um agente de IA um resumo da arquitetura atual do projeto (módulos, dependências, pontos de acoplamento) a partir do código real.
- Discuta e decida, com justificativa baseada nos critérios vistos em aula (não apenas opinião): esse projeto deveria ser mais modular, extrair um serviço, ou está no tamanho certo?

Entregável desta etapa

- ☐ Resumo da arquitetura atual gerado com apoio do agente.
- ☐ Decisão arquitetural registrada, com justificativa.

Etapa 5: Documentação como Contrato - ADR e Diagrama

Objetivo: registrar uma decisão de arquitetura e representá-la visualmente, testando mais de uma abordagem.

Tarefas

12. Escreva **1 ADR** (formato MADR) para uma decisão relevante do projeto, como a da etapa 4.
13. Gere um diagrama (Mermaid, nível C4 de contêiner) da arquitetura atual, pedindo apoio a um agente de IA.
14. Investigue: gere o **mesmo diagrama de uma segunda forma** (ex.: outro prompt com mais contexto, outra ferramenta como Structurizr, ou ajuste manual sobre o resultado da IA) e compare qual versão comunica melhor a arquitetura do seu projeto.

Exemplo de referência (formato do ADR)

```
# 0002-nome-da-decisao.md (formato MADR)
## Contexto
[Qual problema ou situacao motivou esta decisao?]
## Decisao
[O que foi decidido?]
## Consequencias
[O que se ganha e o que se perde com essa decisao?]
```

Entregável desta etapa

- ☐ Arquivo do ADR commitado no repositório (docs/adr/).
- ☐ Diagrama gerado (imagem ou bloco Mermaid) commitado no repositório.
- ☐ Comparação escrita entre as duas versões do diagrama geradas.

Etapa 6: Dívida Técnica, Custo ou Paralelismo

Objetivo: investigar, na prática, **um** dos temas de escala vistos. Escolha UMA das três opções abaixo. A escolha é livre, mas a investigação deve ser real, não apenas descrita.

Opção A: Dívida técnica

- Rode uma ferramenta de análise estática (SonarQube Community Edition, Qodana, ESLint com regras rígidas, ou similar) sobre o seu projeto.
- Identifique 1 sinal real de dívida técnica no código (gerado por IA ou não) e proponha como mitigá-lo.

Opção B: Custo e performance

- Estime quantas chamadas/tokens uma tarefa recente do projeto consumiu (usando logs da ferramenta de IA, se disponíveis, ou uma estimativa justificada).
- Proponha 1 estratégia concreta de otimização (cache de prompt, model routing, batch API) aplicável ao seu contexto, explicando por quê.

Opção C: Agentes em paralelo

- Crie pelo menos 2 git worktrees para 2 tarefas independentes do projeto.
- Tente rodar (ou, na impossibilidade, simule cuidadosamente) agentes em paralelo, um por worktree. Registre o que funcionou e onde houve conflito (portas, dependências, arquivos compartilhados).

Entregável desta etapa

- ☐ Evidência da opção escolhida (print, log, ou arquivo), com um parágrafo explicando o que foi investigado e o que se aprendeu.

Etapa 7: Git e GitHub

Tarefas

15. Publique (ou atualize) o projeto em um repositório no GitHub (público ou privado), com histórico de commits real do processo desta atividade — não um único commit gigante.

Entregável desta etapa

- ☐ Link do repositório com histórico de commits refletindo as Etapas 1 a 6.

Etapa 8: Relatório Final

Objetivo: consolidar as decisões tomadas e as aprendizagens da atividade em um relatório curto.

Roteiro do relatório (máximo 1 página)

16. O nível de autonomia e o guardrail configurados, e o que a comparação da Etapa 1 revelou.
17. O resultado da investigação de TDD/enforcement e da comparação com/sem TDD (Etapa 2).
18. O checkpoint humano definido e o papel assumido (Etapa 3).
19. A decisão arquitetural tomada e como o ADR/diagrama a documentam (Etapas 4 e 5).
20. O que foi investigado na Etapa 6 (Vá Além) e o principal aprendizado.
21. Uma dificuldade real enfrentada durante a atividade - técnica ou conceitual.

Checklist Final de Entregáveis

Confira todos os itens antes de enviar a atividade:

- ☐ Comparação entre os dois modos de autonomia testados (Etapa 1).
- ☐ Hook funcional commitado, com evidência do bloqueio (Etapa 1).
- ☐ Evidência do ciclo Red-Green-Refactor e relato da ferramenta de enforcement investigada (Etapa 2).
- ☐ Comparação entre a tarefa feita com e sem TDD (Etapa 2).
- ☐ Log/transcript da sessão do agente (Etapa 3).
- ☐ Resumo da arquitetura, violação identificada e decisão justificada (Etapa 4).
- ☐ ADR e diagrama commitados, com comparação das duas versões do diagrama (Etapa 5).
- ☐ Evidência da opção escolhida na Etapa 6 (dívida técnica, custo ou paralelismo).
- ☐ Repositório no GitHub com histórico de commits real (Etapa 7).
- ☐ Relatório final de até 1 página cobrindo todos os pontos do roteiro (Etapa 8).